1. Introduction

This report details the design and implementation of a real-time Traffic Light System (TLS) using the STM32F4 Discovery Board running the FreeRTOS kernel. The primary objective of this project is to demonstrate the practical application of real-time operating system concepts—such as multi-tasking, inter-task communication via queues, and software timers—alongside custom middleware developed for hardware abstraction.

The TLS simulates vehicle traffic on a one-way, one-lane road controlled by a single intersection traffic light. The hardware interface consists of three primary components: a potentiometer acting as an analog input to dynamically adjust traffic conditions , three independent LEDs representing the red, amber, and green states of the traffic light , and an array of LEDs driven by daisy-chained 74HC595 shift registers representing the physical presence and movement of cars on the road.

The system is designed to handle dynamic, real-time adjustments. By manipulating the potentiometer, the user dictates the simulated traffic volume. The underlying FreeRTOS application processes this input to scale the rate of new car generation and dynamically alter the duration of the red and green traffic light phases to accommodate the current traffic load. The following sections outline the hardware middleware implementation, the software architecture encompassing task and queue management, and an evaluation of the system's performance and limitations.

2. Design Solution

The design of the Traffic Light System is divided into a hardware middleware layer and an application software layer built on FreeRTOS. The middleware serves as the direct link to the physical board, while the application software uses tasks, queues, and software timers to manage the intersection's logic and car flow.

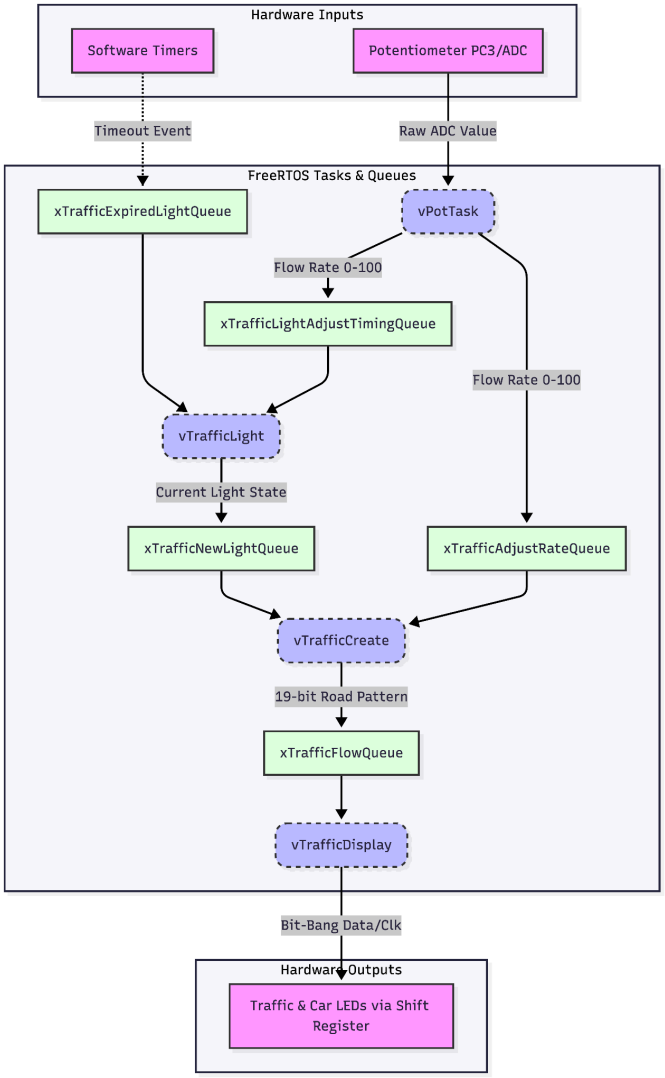


Figure 1: System overview diagram showing the data flow between hardware inputs, FreeRTOS tasks, communication queues, and hardware outputs.

2.1. Middleware and Hardware Setup

All physical hardware components are connected to Port C on the STM32F4 Discovery Board. The middleware initializes these pins so the software can properly interact with them. The traffic light LEDs are controlled by pins PC0 for red, PC1 for amber, and PC2 for green. The daisy-chained shift registers, which control the LEDs representing the cars, use PC6 for the data line, PC7 for the clock pulse, and PC8 for the active-low reset signal. All of these pins are configured as standard push-pull digital outputs. To enable these pins, the AHB1 peripheral clock is activated for Port C.

The potentiometer is connected to pin PC3, which is configured as an analog input with no internal pull-up or pull-down resistors to prevent interference with the voltage readings. This pin maps internally to Channel 13 of the ADC1 peripheral. The ADC is powered by the APB2 peripheral clock and is configured for a 12-bit resolution, providing raw values between 0 and 4095. It operates in single-conversion mode, meaning it only reads the potentiometer voltage exactly when the application software requests it.

In [Table 1] below, a complete list of each Pin is shown, giving the peripheral, direction, and a description of its function. Throughout the design and implementation process, the original two tables are shown in Appendix B, which were used to help layout and organize this project.

Pin	Peripheral	Direction	Function
PC0	GPIOC	Output	Red Traffic Light LED
PC1	GPIOC	Output	Amber Traffic Light LED
PC2	GPIOC	Output	Green Traffic Light LED
PC3	ADC1 Channel 13	Input (Analog)	Potentiometer Input
PC6	GPIOC	Output	Shift Register Data
PC7	GPIOC	Output	Shift Register Clock
PC8	GPIOC	Output	Shift Register Reset (Active-Low)

Table 1: GPIO Pin Layout

A complete electrical schematic of the system is shown in Figure 2. A photograph of the physical circuit implementation is provided in Appendix C for reference.

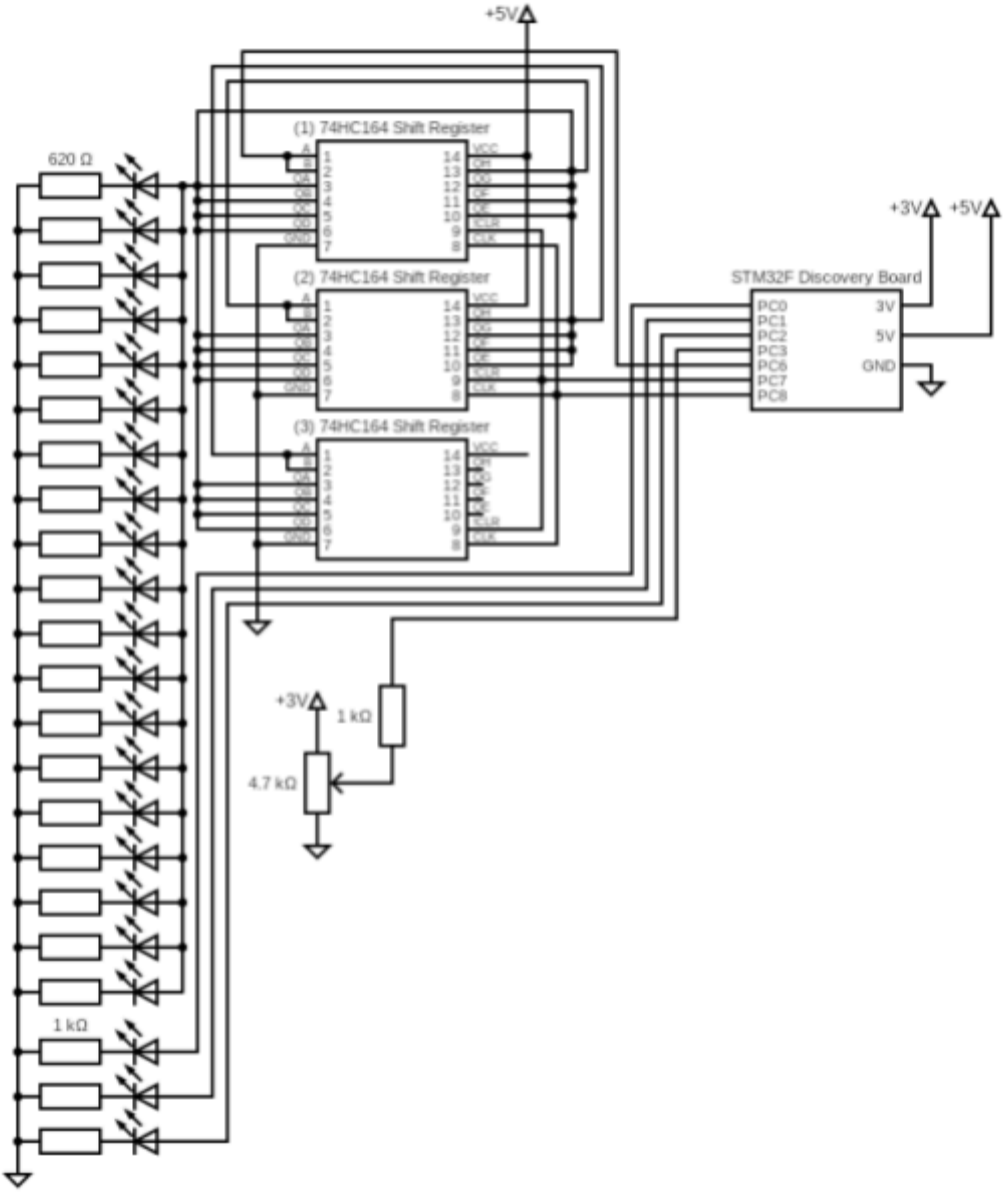


Figure 2: Electrical Schematic of Project 1

2.2. Software Architecture

The software logic is divided into four independent FreeRTOS tasks. To prevent data corruption, these tasks never use global variables to share information; instead, they communicate strictly through FreeRTOS queues. Each task was created using the xTaskCreate() API, with inter-task communication implemented using xQueueCreate() to allocate queues, and xQueueSend() and xQueueReceive() being used for data transfer between tasks.

The traffic flow adjustment task (vPotTask) runs once every second using vTaskDelay() to create a periodic execution cycle. Its job is to trigger an ADC conversion, read the raw potentiometer value, and scale it down to a manageable flow rate between 0 and 100. Updated flow rate values are transmitted using xQueueSend(), and when queues are full, xQueueReceive() is called in a loop to erase old data before inserting the newest value. This queue-draining mechanism ensures that the queues never overflow and that the rest of the system always reacts to the freshest dial position.

The traffic light state task (vTrafficLight) manages the green, amber, and red phases of the intersection. It relies on three FreeRTOS one-shot software timers. When a phase timer expires, it sends a message to an expiration queue. The light task reads this message, checks the most recent flow rate provided by the potentiometer, and calculates the duration for the next phase. Heavy traffic situations dynamically increase the green light duration and decrease the red light duration, while the amber light is kept at a constant two seconds. After updating the physical traffic light LEDs, this task sends its new current colour state to the traffic generation task.

The traffic generation task (vTrafficCreate) is responsible for managing the physical locations of the cars, which are represented by a 19-bit integer. This task runs every 500 milliseconds, establishing a constant visual speed for the cars. During each cycle, it checks the current colour of the traffic light and the current flow rate. If the light is green, the 19-bit integer is simply shifted to the left, moving all cars forward. If the light is red or amber, a custom accumulation function is called. This function splits the road at the intersection line; cars that have already passed the light continue to move forward, while cars behind the light stop and pile up in a line without overwriting one another. Finally, the task uses a random number generator to determine if a brand-new car should spawn at the beginning of the road. The probability of a car spawning is directly tied to the flow rate, with a small offset added to ensure a slow trickle of cars continues even when the potentiometer is turned down to its absolute minimum.

Once the new road pattern is calculated, it is sent to the system display task (vTrafficDisplay). This final task acts as the driver for the physical shift registers. The vTrafficDisplay task waits for new road patterns using a block xQueueReceive() call, placing the task into the blocked state and allowing the scheduler to allocate CPU time to other ready tasks. It first briefly pulls the reset pin low to clear the shift registers of any old data. Then, it loops through the 19-bit integer from the most significant bit down to the least significant bit. For each bit, it sets the data pin either high or low, and then pulses the clock pin to push that specific bit into the shift register. This process repeats until all cars are accurately displayed on the external LEDs.

2.3. FreeRTOS Features Utilized

The project utilizes the following FreeRTOS APIs:

Task Management:

- xTaskCreate() - Creates tasks
- vTaskDelay() - Implements the periodic task execution
- vTaskStartScheduler() - Starts the FreeRTOS kernel

Queue Management

- xQueueCreate() - Initializes communications to the queues
- xQueueSend() - Sends data from a task into a queue
- xQueueReceive() - Receives data from a queue
- uxQueueSpacesAvailable() - Used to prevent queue overflow

Software Timers

- xTimerCreate() - Creates a software timer
- xTimerStart() - Starts the timer
- xTimerChangePeriod() - Modifies traffic light phase durations

3. Discussion

The implementation of the Traffic Light System successfully demonstrates the capabilities of a real-time operating system in managing multiple asynchronous hardware components. By utilizing FreeRTOS tasks, queues, and software timers, the system operates efficiently without blocking the CPU unnecessarily.

3.1. Real-Time Task Management and Communication

To meet the project requirements, the system strictly avoids global variables for inter-task communication, utilizing FreeRTOS queues instead. Queues provide thread-safe data transfers and allow tasks to block (wait) efficiently. For example, the vTrafficDisplay task uses a blocking queue receive (portMAX_DELAY). This means the task consumes zero CPU cycles while waiting for a new road pattern to be generated.

```
xQueueReceive(xTrafficFlowQueue_handle,
              &road_pattern,
              portMAX_DELAY);
```

Figure 3: Some code demonstrating a block xQueueReceive() call using portMAX_delay

A specific design challenge involved the queue length limit. The project specification strictly limited all queues to a length of 5. Because the traffic light task waits up to 10 seconds during a red phase, the timing adjustment queue would naturally fill up with 1-second potentiometer readings. If the queue is full, new readings would be rejected, causing the system to react using stale data. To solve this, a queue-draining mechanism was implemented in the vPotTask and the receiving tasks. The tasks actively receive data in a while loop until the queue is empty, ensuring that the final value applied is always the most recent hardware reading.

3.2. Dynamic Timing Control

Instead of using hardware timers or blocking delay loops, the traffic light phases are implemented using one-shot software timers created with xTimerCreate(). Timers are started using xTimerStart(), and expiration events are communicated to the light task via a queue from the timer callback function. The task then calculates the next phase's duration dynamically. The math ensures that as the traffic flow rate increases (0-100), the green light time scales up (5000ms + 50 * flowRate), and the red light time scales down (10000ms - 50 * flowRate). The amber light remains constant at 2000ms.

3.3 Traffic Generation Algorithm

The traffic generation algorithm (vTrafficCreate) is responsible for moving the cars and generating new ones. The logic executes every 500 milliseconds, forcing the cars to move at a consistent speed of two LEDs per second.

When the light is green, the 19-bit road_pattern integer is shifted left by 1, simulating forward movement. When the light is red or amber, the system applies a custom bitwise masking function. It isolates the top 12 bits (cars past the intersection) using the mask 0x07F800 and shifts them forward. It isolates the bottom 11 bits (cars before the intersection) using the mask 0x0007FF and evaluates them individually to create a pile-up effect without cars overwriting each other.

To generate new cars, a random number between 0 and 99 is generated. If this number is less than the current flow rate plus 10, a new car is added to the start of the road (the least significant bit). The + 10 offset ensures that even when the potentiometer is at 0 (minimum traffic), there is still a small, continuous 10% chance of a car spawning, fulfilling the "low traffic" requirement.

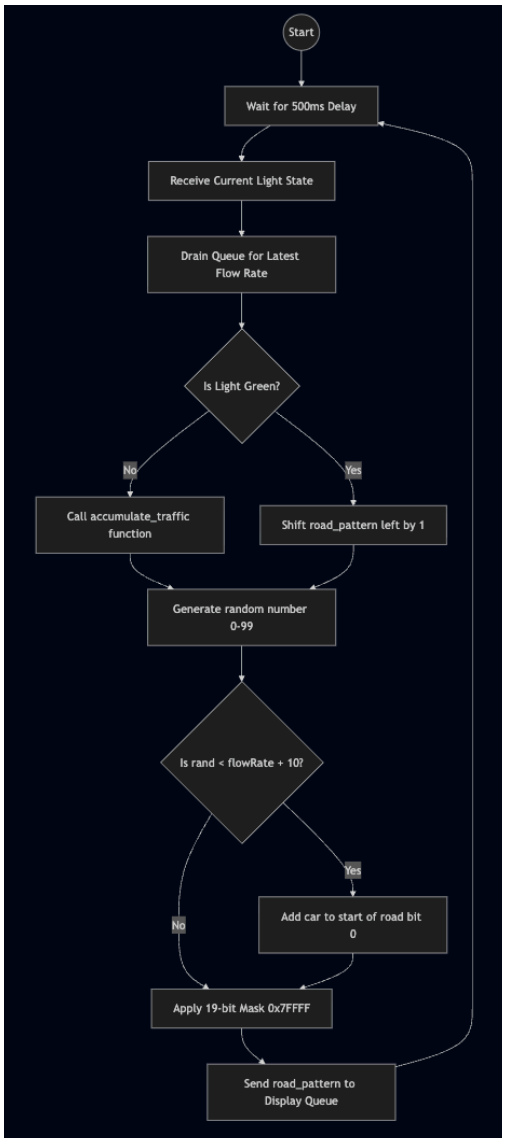


Figure 4: Flow chart of the traffic-generating algorithm.

3.4 System Display Algorithm

The system display algorithm (vTrafficDisplay) translates the 19-bit software integer into physical LED outputs via the 74HC595 shift registers. Because the shift registers push data sequentially, the algorithm isolates and evaluates each bit of the 19-bit integer starting from the Most Significant Bit (bit 18) down to the Least Significant Bit (bit 0). Depending on whether the bit is a 1 or a 0, the data pin is set appropriately. A clock pulse is then manually generated by setting the clock pin HIGH and then LOW to push the bit into the register. Before shifting new data, the reset pin is toggled to clear the previous state.

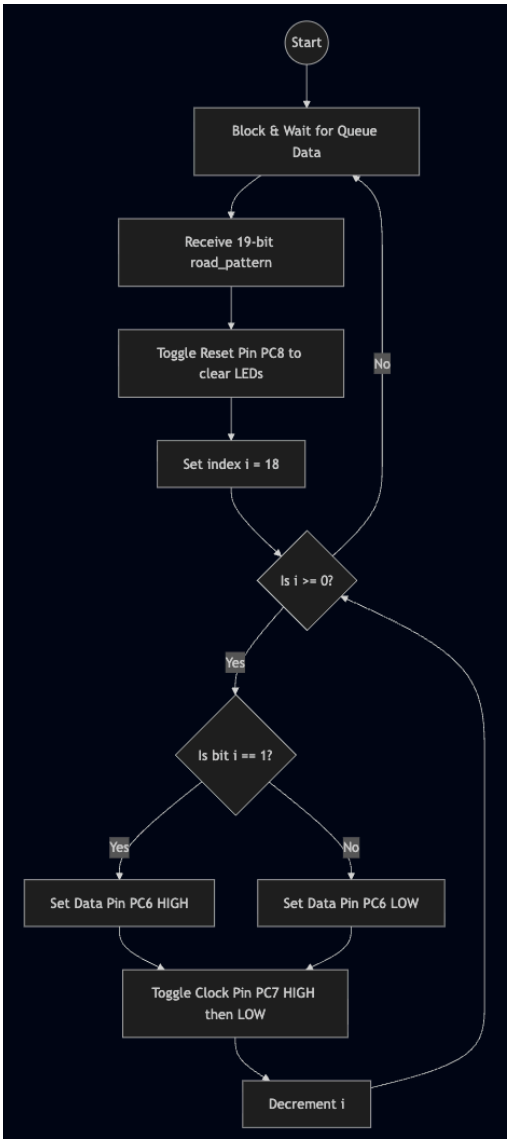


Figure 5: Flow chart of the system display algorithm.

4. Limitations and Possible Improvements

While the current implementation of the Traffic Light System meets all functional and technical requirements, there are inherent limitations in the hardware utilization and software design. Addressing these limitations provides several opportunities for system optimization.

4.1 System Limitations

1. **Queue Management Overhead:** The project specifications required all queues to be initialized with a length of 5. Because the potentiometer task updates the flow rate every 1 second, but the traffic light task can wait up to 10 seconds during a red phase, the queue rapidly fills up. To prevent the system from using stale data, a queue-draining loop was implemented to discard older messages. While this ensures the data is fresh, it creates unnecessary RTOS overhead by constantly enqueuing and dequeuing data that will never be used.
2. **Bit-Banging the Shift Register:** In the vTrafficDisplay task, the data and clock signals are manually toggled (bit-banged) using GPIO commands to push data into the 74HC595 shift registers. To ensure the physical hardware registers the signals, small blocking for loops (e.g., for(volatile int d=0; d<10; d++);) are used to create microsecond delays. These blocking loops waste CPU cycles that could otherwise be allocated to lower-priority tasks or the FreeRTOS idle task.
3. **ADC Polling:** The vPotTask initiates an ADC conversion and then actively blocks the CPU in a while loop, waiting for the End-of-Conversion (EOC) flag to be set. This active polling mechanism is somewhat inefficient, as it forces the CPU to wait on the slower analog hardware rather than performing other computations or entering a low-power state.
4. **Scalability of the Road Representation:** The physical position of the cars is currently stored within a single 32-bit unsigned integer (uint32_t), utilizing 19 bits to represent the 19 physical LEDs on the shift registers. If the physical system were expanded to simulate a longer road (e.g., greater than 32 LEDs) or multiple lanes, this single-integer approach would overflow and fail.

4.2. Possible Improvements

1. **Implementing "Mailbox" Queues:** If the queue length constraint of 5 were lifted, the xTrafficLightAdjustTimingQueue and xTrafficAdjustRateQueue could be initialized with a length of exactly 1. This would allow the use of the FreeRTOS xQueueOverwrite() API. A mailbox queue automatically overwrites its single item with the newest data, eliminating the CPU overhead caused by the current queue-draining while loops.
2. **Hardware SPI (Serial Peripheral Interface):** To eliminate the inefficient bit-banging and blocking delays in the display task, the software could be updated to use the STM32F4's dedicated hardware SPI peripheral. By configuring the SPI to transmit data to the shift registers, the CPU would simply write the 19-bit pattern to an SPI data register, and the hardware would automatically handle the clock pulsing and data shifting at high speeds without any blocking software loops.
3. **Interrupt-Driven ADC or DMA:** The ADC reading process could be significantly improved by utilizing interrupts or Direct Memory Access (DMA). Instead of polling the EOC flag, the task could trigger the conversion and then enter a blocked state. An interrupt service routine (ISR) would then wake the task once the conversion is complete, freeing up the CPU for other operations during the conversion time.
4. **Data Structure Expansion:** To make the system scalable for larger roads, the road_pattern could be refactored from a single integer into a dynamic array of booleans or a custom struct. This would allow the system to simulate a theoretically infinite number of cars and LED positions, limited only by the available RAM on the STM32F4.

5. Summary

The Traffic Light System project successfully demonstrated the design and implementation of a real-time embedded application using the STM32F4 Discovery Board and the FreeRTOS kernel. The primary objective was to simulate a dynamic, single-lane traffic intersection by integrating custom hardware middleware with real-time task scheduling and inter-task communication.

At the hardware layer, custom middleware was developed to configure and manage the microcontroller's peripherals. This included setting up General Purpose Input/Output (GPIO) pins to drive the traffic light LEDs and a daisy-chained 74HC595 shift register network, which visually represented the physical presence and movement of cars on the road. Additionally, the Analog-to-Digital Converter (ADC) was configured to read real-time inputs from a potentiometer, allowing the user to simulate varying levels of traffic volume dynamically.

At the software layer, the system architecture was strictly modular, utilizing four distinct FreeRTOS tasks to separate the logic for reading the analog input, controlling the traffic light states, generating the traffic array, and driving the LED display. To ensure thread-safe operation and prevent data corruption, all data was passed between these tasks using five dedicated FreeRTOS queues, completely avoiding the use of global variables. FreeRTOS one-shot software timers were successfully deployed to manage the state machine of the traffic lights without relying on inefficient blocking delays.

Overall, the final implementation met all functional and technical requirements specified in the project guidelines. The system successfully responds to real-time user inputs to dynamically scale both the frequency of new cars appearing and the durations of the green and red light phases. Through careful bitwise manipulation, the cars accurately move, stop, and accumulate at the intersection based on the current state of the light. While future iterations could optimize the system further by integrating hardware SPI or DMA, the current design stands as a robust, highly responsive, and reliable real-time embedded system.

6. Appendix

Appendix A - Source Code

The following source code contains the complete implementation of the Traffic Light System (TLS) in main.c. This includes the custom hardware middleware (GPIO and ADC initialization), FreeRTOS task definitions, queue management, software timer callbacks, and the bitwise traffic generation logic.

```
/* Standard includes. */
#include <stdint.h>
#include <stdio.h>
#include "stm32f4_discovery.h"
#include "stm32f4xx_adc.h"
#include "stm32f4xx_gpio.h"
#include <stdlib.h>
#include <time.h>

/* Kernel includes. */
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"

/*-----------------------------------------------------------*/
#define mainQUEUE_LENGTH 100

#define red_led     GPIO_Pin_0
#define amber_led   GPIO_Pin_1
#define green_led   GPIO_Pin_2
#define POT_PIN     GPIO_Pin_3

#define DATA_PIN    GPIO_Pin_6
#define CLOCK_PIN   GPIO_Pin_7
#define RESET_PIN   GPIO_Pin_8

#define YES         1
#define NO          0

#define red         0x01
#define amber       0x02
```

```
#define green       0x04
#define all         0x07

//Functions
void ADC_init(void);
void GPIOC_init(void);
void Queue_init(void);

//Task Prototypes
void vPotTask (void *pvParameters);

//Traffic Tasks Prototypes
void vTrafficFlowTask (void *pvParameters);
void vTrafficLightAdjustmentTask (void *pvParameters);
void vTrafficLight (void *pvParameters);
void vTrafficCreate (void *pvParameters);
void vTrafficDisplay (void *pvParameters);

// Timer Call backs for traffic light phases
void vCallBackGreen(TimerHandle_t xTimer);
void vCallBackAmber(TimerHandle_t xTimer);
void vCallBackRed(TimerHandle_t xTimer);

// Helper function for accumulating traffic
void accumulate_traffic(uint32_t *road_pattern);

//Queues
xQueueHandle xTrafficFlowQueue_handle;
xQueueHandle xTrafficNewLightQueue_handle;
xQueueHandle xTrafficExpiredLightQueue_handle;
xQueueHandle xTrafficAdjustRateQueue_handle;
xQueueHandle xTrafficLightAdjustTimingQueue_handle;

//Software Timer Definitions
TimerHandle_t xTimer_Green;
TimerHandle_t xTimer_Amber;
TimerHandle_t xTimer_Red;

static void prvSetupHardware( void );

/*-----------------------------------------------------------*/
TaskHandle_t controlTaskHandle = NULL;

int main(void)
```

```
{
   prvSetupHardware();

   ADC_init();     //Initialize ADC
   GPIOC_init();   //Initialize Port C
   Queue_init();   //Initialize Queues and Timers

   srand(time(NULL));  //completely randomizes the rand() function seed ensuring a random
sequence whenever ran

   xTaskCreate(vPotTask,"Pot",128,NULL,1,NULL);

   xTaskCreate(vTrafficLight,"TrafficLight",128,NULL,1,NULL);
   xTaskCreate(vTrafficCreate,"TrafficCreate",128,NULL,1,NULL);
   xTaskCreate(vTrafficDisplay,"TrafficDisplay",128,NULL,1,NULL);

   vTaskStartScheduler();  //Allows tasks to be run

   return 0;
}

/*-----------------------------------------------------------*/

void GPIOC_init(void){

   /* Enable clock for GPIOC peripheral*/
   RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);

   GPIO_InitTypeDef GPIO_InitStruct;

   GPIO_InitStruct.GPIO_Pin =    red_led | amber_led | green_led | DATA_PIN | CLOCK_PIN |
RESET_PIN;
   GPIO_InitStruct.GPIO_Mode =   GPIO_Mode_OUT;
   GPIO_InitStruct.GPIO_Speed =  GPIO_Speed_50MHz;
   GPIO_InitStruct.GPIO_OType =  GPIO_OType_PP;
   GPIO_InitStruct.GPIO_PuPd =   GPIO_PuPd_NOPULL;
   GPIO_Init(GPIOC, &GPIO_InitStruct);

   //Pot Pin Initialization
   GPIO_InitStruct.GPIO_Pin =    POT_PIN;
   GPIO_InitStruct.GPIO_Mode =   GPIO_Mode_AN;
   GPIO_InitStruct.GPIO_PuPd =   GPIO_PuPd_NOPULL;
   GPIO_Init(GPIOC, &GPIO_InitStruct);
```

```
  GPIO_ResetBits(GPIOC, RESET_PIN);
  for(int d = 0; d<10000; d++);

  //Enable Shift Register
  GPIO_SetBits(GPIOC, RESET_PIN);

  //Set Data Line HIgh
  GPIO_SetBits(GPIOC, DATA_PIN);

  //Initialization of Traffic LEDs (ensuring they are all plugged in correctly)
  for(int i = 0; i < 24; i++){

        GPIO_SetBits(GPIOC, CLOCK_PIN);

        for(int j = 0; j < 1000000; j++);

        GPIO_ResetBits(GPIOC, CLOCK_PIN);

        for(int j = 0; j < 1000000; j++);
     }

  GPIO_ResetBits(GPIOC, RESET_PIN);
  for(int d = 0; d<10000; d++);
}

void ADC_init(void)
{
 // Enable clock for ADC1
 RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);

 // Initialize ADC Structure
 ADC_InitTypeDef ADC_InitStruct;

 ADC_InitStruct.ADC_ContinuousConvMode =      DISABLE;
 ADC_InitStruct.ADC_DataAlign          =      ADC_DataAlign_Right;
 ADC_InitStruct.ADC_Resolution         =      ADC_Resolution_12b;
 ADC_InitStruct.ADC_ScanConvMode       =      DISABLE;
 ADC_InitStruct.ADC_ExternalTrigConv   =      DISABLE;
 ADC_InitStruct.ADC_ExternalTrigConvEdge =    DISABLE;

 ADC_Init(ADC1, &ADC_InitStruct);
 ADC_Cmd(ADC1, ENABLE);

 // Configure Channel 13 (PC3) with a sample time
```

```
  ADC_RegularChannelConfig(ADC1, ADC_Channel_13, 1, ADC_SampleTime_144Cycles);
}

void Queue_init(void){
   //Queues
   xTrafficFlowQueue_handle =              xQueueCreate( 5, sizeof(uint32_t) );    //19-bit car
pattern
   xTrafficNewLightQueue_handle =          xQueueCreate( 5, sizeof(uint8_t) );     //Notifies of a
new light state
   xTrafficExpiredLightQueue_handle =      xQueueCreate( 5, sizeof(uint8_t) );     //Notifies that
the current phase timer has expired
   xTrafficAdjustRateQueue_handle =        xQueueCreate( 5, sizeof(uint16_t) );    //pot value to
control car gen frequency  [correction]
   xTrafficLightAdjustTimingQueue_handle = xQueueCreate( 5, sizeof(uint16_t) );
//potentiometer value to control light phase duration [correction]

   vQueueAddToRegistry(xTrafficFlowQueue_handle, "TrafficFlowQueue");
   vQueueAddToRegistry(xTrafficNewLightQueue_handle, "TrafficNewLightQueue");
   vQueueAddToRegistry(xTrafficExpiredLightQueue_handle, "TrafficExpiredLightQueue");
   vQueueAddToRegistry(xTrafficAdjustRateQueue_handle, "TrafficAdjustRateQueue");
   vQueueAddToRegistry(xTrafficLightAdjustTimingQueue_handle,
"TrafficLightAdjustTimingQueue");

   // Software Timer Definitions
   xTimer_Green  = xTimerCreate("TMR1", pdMS_TO_TICKS(10000), pdFALSE, 0,
vCallBackGreen);
   xTimer_Amber  = xTimerCreate("TMR2", pdMS_TO_TICKS(2000),  pdFALSE, 0,
vCallBackAmber);
   xTimer_Red    = xTimerCreate("TMR3", pdMS_TO_TICKS(7000),  pdFALSE, 0,
vCallBackRed);
}

void vPotTask(void *pvParameters){
   uint16_t potValue;
   uint16_t flowRate;
   uint16_t dummy;

   const TickType_t xDelay = pdMS_TO_TICKS(1000); // 20 Hz, changed to 1 second --> Step 6
of the manual

   for (;;)
   {
      // Start conversion
      ADC_SoftwareStartConv(ADC1);
```

```
        // Wait for conversion complete
        while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));

        // Read ADC value
        potValue = ADC_GetConversionValue(ADC1);

        // Convert potValue to a 0-100 value that can be easily used
        flowRate = (potValue * 100) / 0xFFF;

        // Ensure there is space in the queue so we don't drop the freshest data!
        if(uxQueueSpacesAvailable(xTrafficAdjustRateQueue_handle) == 0) {
            xQueueReceive(xTrafficAdjustRateQueue_handle, &dummy, 0);
        }
        if(uxQueueSpacesAvailable(xTrafficLightAdjustTimingQueue_handle) == 0) {
            xQueueReceive(xTrafficLightAdjustTimingQueue_handle, &dummy, 0);
        }

        xQueueSend(xTrafficAdjustRateQueue_handle, &flowRate, 0);   // No blocking
        xQueueSend(xTrafficLightAdjustTimingQueue_handle, &flowRate, 0);   // No blocking

        vTaskDelay(xDelay);
    }
}

void vTrafficLight (void *pvParameters){
    uint8_t current_light = green;
    uint8_t expired_light;
    uint16_t flowRate = 50;

    // Initial state: start with Green light
    xTimerStart(xTimer_Green, 0);
    xQueueSend(xTrafficNewLightQueue_handle, &current_light, 0);

    for (;;) {
        // Update physical LEDs
        GPIO_ResetBits(GPIOC, red_led | amber_led | green_led);
        if (current_light == red) GPIO_SetBits(GPIOC, red_led);
        else if (current_light == amber) GPIO_SetBits(GPIOC, amber_led);
        else if (current_light == green) GPIO_SetBits(GPIOC, green_led);

        // Wait for a timer to expire
        if (xQueueReceive(xTrafficExpiredLightQueue_handle, &expired_light, portMAX_DELAY)) {
```

```
      // Peek at latest potentiometer value to adjust timings
      // Get latest flow rate and empty the queue to prevent it from filling up
      uint16_t temp_flow_light;

      while (xQueueReceive(xTrafficLightAdjustTimingQueue_handle, &temp_flow_light, 0) ==
pdPASS){
          flowRate = temp_flow_light; // Keeps the most recent value
      }
          if (expired_light == green) {
              current_light = amber;
              xTimerChangePeriod(xTimer_Amber, pdMS_TO_TICKS(2000), 0); // Amber is
constant 2s
          }
          else if (expired_light == amber) {
              current_light = red;
              // Red time inversely proportional to flow
              uint32_t red_time = 10000 - (50 * flowRate);         //sets time
             xTimerChangePeriod(xTimer_Red, pdMS_TO_TICKS(red_time), 0);       //runs the set
time
          }
          else if (expired_light == red) {
              current_light = green;
              // Green time proportional to flow
              uint32_t green_time = 5000 + (50 * flowRate);      //sets time
              xTimerChangePeriod(xTimer_Green, pdMS_TO_TICKS(green_time), 0);          //runs
the set time
          }

          // Notify Create_Traffic task of the new light state
          xQueueSend(xTrafficNewLightQueue_handle, &current_light, 0);
      }
  }
}
void vTrafficCreate (void *pvParameters){
  uint32_t road_pattern = 0; // 19-bit road representation
  uint8_t current_light = green;
  uint16_t flowRate = 50;

  for (;;) {
      // Check if light state changed (non-blocking)
      xQueueReceive(xTrafficNewLightQueue_handle, &current_light, 0);

      // Get latest flow rate (probability of new car)
      uint16_t temp_flow_create;
```

```
    while (xQueueReceive(xTrafficAdjustRateQueue_handle, &temp_flow_create, 0) ==
pdPASS) {
        flowRate = temp_flow_create; // Keeps the most recent value
    }

    if (current_light == green) {
        // Move cars forward
        road_pattern <<= 1;
    } else {
        // Amber or Red: Pile up cars before the intersection
            accumulate_traffic(&road_pattern);
    }

    // Randomly add a new car based on traffic flow rate
    // rand() % 100 generates 0-99. flowRate is 0-100.
    // Ensure that at 0 some traffic will still flow (+ 10)
    if ((rand() % 100) < ((flowRate) + 10)) {
        road_pattern |= 1; // Add car to start of the road (LSB)
    }

    // Keep it strictly 19 bits
    road_pattern &= 0x7FFFF;

    // Send to display task
    xQueueSend(xTrafficFlowQueue_handle, &road_pattern, 0);

    vTaskDelay(pdMS_TO_TICKS(500)); // Determines car speed (update rate) (500 2/s)
  }
}

// Helper to pile up cars at red/amber light
void accumulate_traffic(uint32_t *road) {
   uint32_t temp_road = *road;

   // Stop line is between LED 10 and LED 11 (bits 10 and 11)
   uint32_t after_intersection = temp_road & 0x7FF800; // Top 12 bits (past the light)
   uint32_t before_intersection = temp_road & 0x007FF; // Bottom 11 bits (before light)

   // Cars after the light keep moving forward
   after_intersection <<= 1;

   // Cars before the light pile up (shift only if the spot ahead is empty)
   uint32_t new_before = 0;
```

```
    // Loop through the bottom 11 bits (Index 10 down to 0)
    for(int i = 10; i >= 0; i--) {
        if (before_intersection & (1 << i)) {
            if (i < 10 && ((new_before & (1 << (i+1))) == 0)) {
                new_before |= (1 << (i+1)); // Move forward into empty space
            } else {
                new_before |= (1 << i); // Stop and pile up at the line
            }
        }
    }

    // Recombine the road
    *road = (after_intersection & 0x07F800) | (new_before & 0x007FF);
}


void vTrafficDisplay (void *pvParameters){
   uint32_t road_pattern = 0;

   for (;;) {
      if (xQueueReceive(xTrafficFlowQueue_handle, &road_pattern, portMAX_DELAY)) {

         // Clear shift register
         GPIO_ResetBits(GPIOC, RESET_PIN);
         for(volatile int d=0; d<10; d++);
         GPIO_SetBits(GPIOC, RESET_PIN);

         // Shift out the 19 bits (MSB to LSB)
         for (int i = 18; i >= 0; i--) {
            if (road_pattern & (1 << i)) {
               GPIO_SetBits(GPIOC, DATA_PIN);
            } else {
               GPIO_ResetBits(GPIOC, DATA_PIN);
            }

            // Clock pulse
            GPIO_SetBits(GPIOC, CLOCK_PIN);
            for(volatile int d=0; d<10; d++); // Short delay
            GPIO_ResetBits(GPIOC, CLOCK_PIN);
         }
      }
   }
}
```

```
/*-----------------------------------------------------------*/
/* Timer Callbacks (Send expired light state to queue) */

void vCallBackGreen(TimerHandle_t xTimer) {
  uint8_t expired = green;
  xQueueSend(xTrafficExpiredLightQueue_handle, &expired, 0);
}

void vCallBackAmber(TimerHandle_t xTimer) {
  uint8_t expired = amber;
  xQueueSend(xTrafficExpiredLightQueue_handle, &expired, 0);
}

void vCallBackRed(TimerHandle_t xTimer) {
  uint8_t expired = red;
  xQueueSend(xTrafficExpiredLightQueue_handle, &expired, 0);
}

/*-----------------------------------------------------------*/

void vApplicationMallocFailedHook( void )
{
  /* The malloc failed hook is enabled by setting
  configUSE_MALLOC_FAILED_HOOK to 1 in FreeRTOSConfig.h.

  Called if a call to pvPortMalloc() fails because there is insufficient
  free memory available in the FreeRTOS heap.  pvPortMalloc() is called
  internally by FreeRTOS API functions that create tasks, queues, software
  timers, and semaphores.  The size of the FreeRTOS heap is set by the
  configTOTAL_HEAP_SIZE configuration constant in FreeRTOSConfig.h. */
  for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationStackOverflowHook( xTaskHandle pxTask, signed char *pcTaskName )
{
  ( void ) pcTaskName;
  ( void ) pxTask;

  /* Run time stack overflow checking is performed if
  configconfigCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2.  This hook
  function is called if a stack overflow is detected.  pxCurrentTCB can be
  inspected in the debugger if the task name passed into this function is
  corrupt. */
```

```
  for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationIdleHook( void )
{
volatile size_t xFreeStackSpace;

  /* The idle task hook is enabled by setting configUSE_IDLE_HOOK to 1 in
  FreeRTOSConfig.h.

  This function is called on each cycle of the idle task.  In this case it
  does nothing useful, other than report the amount of FreeRTOS heap that
  remains unallocated. */
  xFreeStackSpace = xPortGetFreeHeapSize();

  if( xFreeStackSpace > 100 )
  {
     /* By now, the kernel has allocated everything it is going to, so
     if there is a lot of heap remaining unallocated then
     the value of configTOTAL_HEAP_SIZE in FreeRTOSConfig.h can be
     reduced accordingly. */
  }
}
/*-----------------------------------------------------------*/

static void prvSetupHardware( void )
{
  /* Ensure all priority bits are assigned as preemption priority bits.
  http://www.freertos.org/RTOS-Cortex-M3-M4.html */
  NVIC_SetPriorityGrouping( 0 );

  /* TODO: Setup the clocks, etc. here, if they were not configured before
  main() was called. */
}
```

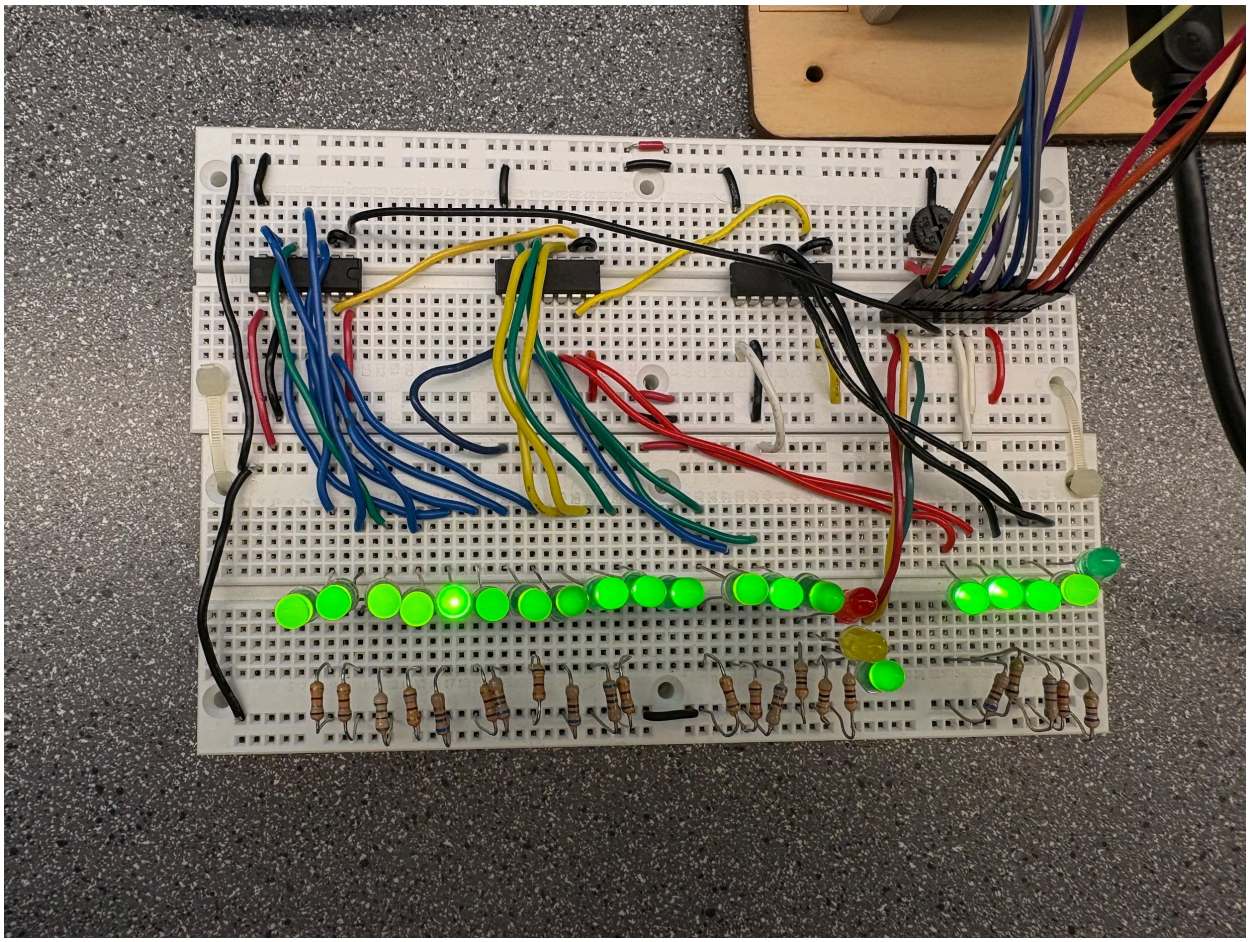
Appendix B - Original Design/Implementation Pinout Tables

The tables included in this appendix were developed during the initial planning and hardware layout phase of Project 1. The first table reflects the original pin configuration outline provided in the project specification. The second table was created by the team to assist with connecting the STM32F4 Discovery Board pins to the breadboard connections during physical implementation. The second table uses the “BPX” (Breadboard Port X) notation to identify the corresponding DIP connector pins used in the hardware setup.

STM32F0 Port Pin	Signal	Notes
PC0	Red Light	Traffic Light
PC1	Amber Light	Traffic Light
PC2	Green Light	Traffic Light
		Traffic flow shift register
PC8	Shift Register Reset	Active Low, minimum 1us period
PC7	Shift Register Clock	Falling edge trigger
PC6	Shift Register Data	After clock falling edge data hold time of a minimum of 1 us
PC3	Potentiometer input	0 – 3 Volt Input

Pinout for Breadboard:

Breadboard Port	Signal	STM32F0 Port Pin
BP0	Red Light	PC0
BP1	Amber Light	PC1
BP2	Green Light	PC2
BP3	Potentiometer	PC3
BP4	Shift Reg Data	PC6
BP5	Shift Reg Clock	PC7
BP6	Shift Reg Reset	PC8
BP7	3V	
BP8	5V	
BP9	GND	

Appendix C - Physical Circuit Implementation

Appendix C shows the assembled hardware implementation on the breadboard and STM32F4 Discovery board.